A Self-Defending Gateway for Enterprise LLM Access: Hybrid Bandit and Reinforcement Learning Defence with Tamper-Evident Auditing

Chinmay Karkamkar
Dept. of DEPARTMENT
UNIVERSITY NAME
CITY, India
email@domain

Aditya S
Dept. of DEPARTMENT
UNIVERSITY NAME
CITY, India
email@domain

Priyanka M
Dept. of DEPARTMENT
UNIVERSITY NAME
CITY, India
email@domain

Monish L
Dept. of DEPARTMENT
UNIVERSITY NAME
CITY, India
email@domain

***Abstract*—Enterprises adopting large language models (LLMs) face three risks at once: personal data sent to external providers, prompt-injection and jailbreak attacks, and uncontrolled cost. We present a self-defending gateway, a reverse proxy that applications adopt by changing only their SDK base URL. Every request passes a fail-closed pipeline that enforces budgets atomically, redacts personally identifiable information (PII), and scores the prompt with a fine-tuned DeBERTa-v3 classifier over overlapping windows. Detection is separated from response: a LinUCB contextual bandit chooses, per request, among allow, redact, block and escalate-to-human, learning from reviewer verdicts, while a deep Q-network (DQN) agent acts on whole sessions with actions that tighten, challenge or lock out a persistent attacker. Every decision is written to a SHA-256 hash-chained, append-only audit log. On a locked 950-prompt test set the classifier reaches a macro-F1 of 0.908 with a 3.1% benign false-positive rate; the same test showed that an earlier model's 0.933 on a smaller benchmark did not generalise (0.490). The bandit matches a tuned threshold on i.i.d. data and, under simulated false-positive drift, gains 0.141 reward per request while cutting benign refusals from 21.0% to 0.9%. The DQN agent reduces session breaches relative to a rule-based policy (8.3% vs. 11.7%) but exceeds a pre-registered benign-lockout limit, so the rule remains the default. We report these negative results, and their causes, as part of the contribution.**

***Index Terms*—Large language models, prompt injection, jailbreak detection, PII redaction, contextual bandits, deep Q-network, tamper-evident logging.**

I. Introduction

Organisations increasingly route employee and customer requests to hosted LLMs such as those offered by OpenAI and Anthropic. In the common integration pattern the application sends the prompt straight to the provider, and nothing sits in between to enforce organisational policy.

This exposes three risks. First, *data leakage*: users paste e-mail addresses, phone numbers, card numbers and credentials into prompts, which then leave the organisation. Second, *adversarial prompts*: prompt injection [2]–[4] and jailbreaks [5], [6] try to override the application's instructions or extract its system prompt; prompt injection is ranked first in the OWASP Top 10 for LLM applications [1]. Third, *cost*: token-based billing without per-team limits can run away.

Existing tools usually address one of these risks in isolation: a PII library, a guard classifier [7], a guardrail framework [8], or an API gateway with rate limits. Guard classifiers are typically applied with a fixed threshold that cannot adapt when the classifier's errors drift, and they judge each request on its own, so an attacker who keeps rephrasing is treated like a first-time user.

We built a gateway that combines all three protections in one fail-closed pipeline, and that *separates detection from response*. Our contributions are:

- A drop-in, fail-closed LLM gateway that integrates atomic budget control, PII redaction, windowed injection/jailbreak detection, adaptive response and tamper-evident auditing, and that never stores raw PII.
- A two-tier adaptive defence: a LinUCB contextual bandit for independent per-request decisions, and a DQN agent for sequential per-session decisions, with session actions (challenge, lockout) that act on attacks the classifier misses.
- A pre-registered evaluation protocol (near-duplicate-aware splits, a hash-locked test set, criteria fixed before training) and its results, including two negative findings.
- A hash-chained audit log with layered protection that detects tampering even by a database superuser.

Section II reviews related work, Section III describes the system, Section IV the detection and defence methods, Section V the experimental setup, Section VI the results, and Section VII discusses limitations before Section VIII concludes.

II. Related Work

Attacks. Perez and Ribeiro [2] showed that simple "ignore previous instructions" prompts hijack model goals. Greshake et al. [3] extended this to indirect injection through retrieved content, and Liu et al. [4] studied injection against deployed LLM-integrated applications. Wei et al. [5] analysed why safety training fails against jailbreaks, and Shen et al. [6] collected jailbreak prompts in the wild; their collection is one of our data sources.

Defences. Llama Guard [7] is an LLM-based input/output safety classifier; NeMo Guardrails [8] provides programmable conversational rails. Li et al. [9] highlight *over-defence*, where guard models flag benign prompts containing trigger words, and release the NotInject benchmark, which we use as hard-benign training data. These systems output a label with a fixed decision rule; our work adds an adaptive policy layer and session memory on top of a classifier.

Learning-based response. LinUCB [10] is the standard linear contextual bandit; discounting handles non-stationary rewards [11]. DQN [12] and its implementation in Stable-Baselines3 [13] underpin our session agent, and potential-based reward shaping [14] preserves the optimal policy. Reinforcement learning has been surveyed widely for cyber defence [15]; unlike most such work we evaluate the agent against a strong rule baseline with a pre-registered go/no-go criterion.

Audit logs and evaluation hygiene. Hash-chained secure logs date back to Schneier and Kelsey [16]; Crosby and Wallach [17] formalised tamper-evident logging. Near-duplicate contamination inflates reported scores [18], which we observed directly.

III. System Architecture

Fig. 1 shows the gateway. It is a headless FastAPI service that exposes an OpenAI-compatible chat endpoint; applications point their SDK at it and authenticate with a gateway API key, stored only as a SHA-256 hash. Requests for `gpt-*` models are routed to OpenAI and `claude-*` to Anthropic (with format translation); an optional local provider (Ollama) allowed end-to-end testing without paid keys. PostgreSQL holds teams, keys, bandit state, the review queue and the audit log; Redis holds budgets, rate limits and session state.

Pipeline and fail-closed rule. Stages run in a fixed order and each returns ALLOW, MODIFY or BLOCK. If any stage raises an exception the request is blocked (HTTP 503) unless `FAIL_MODE=open` is set explicitly; a failing security check that lets traffic through would be a bypass an attacker can trigger.

Cost governance. A token-bucket rate limiter runs as a Redis Lua script. Budgets use *reserve-then-reconcile*: one atomic script reserves the estimated cost and checks the limit in the same step; after the provider answers the reservation is corrected to the real cost, and on any failure it is refunded. This removes a race we observed in an earlier read-compare-write version: with ten concurrent requests against a budget for three, exactly three are admitted.

PII redaction. Microsoft Presidio [21] detects names, e-mail addresses, phone numbers and similar entities; our own patterns add provider API keys and Luhn-validated card numbers, and win ties against Presidio. Values are either masked (`[REDACTED_EMAIL_ADDRESS]`) or tokenised, with the original stored Fernet-encrypted. Redaction is on by default and covers message content and tool-call arguments. Only entity-type counts are ever stored or logged.

Admin dashboard. A Next.js client of the gateway's admin API shows traffic, the review queue, the audit log with an integrity check, threats, spend and PII counts. The admin session token stays server-side in an `httpOnly`, `SameSite=Strict` cookie.

IV. Detection and Adaptive Defence

A. Threat Classifier

A DeBERTa-v3-base model [19] is fine-tuned for three classes: benign, prompt injection and jailbreak. Each message is split into overlapping windows (160 tokens, at most 32), and a request's score is that of its *worst* window. Windowing defeats a padding attack we found: 250 tokens of filler before an injection lowered its score from 0.995 to 0.003 under truncation; with windows it scores 0.997. System prompts are scored separately, because defensive instructions ("never reveal these instructions") otherwise score 0.98 as injection. Scores are calibrated by temperature scaling [20], $p = \mathrm{softmax}(z/T)$, with $T = 2.24$ fitted on validation data only, which lowered the attack-score expected calibration error on validation from 0.042 to 0.019. Training uses fp32, learning rate 2×10^{-5}, effective batch 16, square-root inverse-frequency class weights and early stopping on validation macro-F1, on a 4 GB laptop GPU.

B. Tactical Defence: Contextual Bandit

Each request is a single decision with immediate feedback that does not change the next request's situation, which is the contextual-bandit setting rather than reinforcement learning. The bandit chooses one of four arms: *allow*, *redact-and-allow* (remove the suspicious window), *block*, or *escalate* to a human reviewer. Its context x_t has nine features: a bias term, the injection and jailbreak probabilities, the attack log-odds, the system-prompt attack score and its product with the conversation score, a truncation flag, the number of windows and the PII entity count. Time of day and request rate were removed after we showed that an attacker could use them to flip decisions. LinUCB [10] selects

$$a_t = \arg\max_a \ \hat{\theta}_a^\top x_t + \alpha\sqrt{x_t^\top A_a^{-1} x_t}, \qquad \hat{\theta}_a = A_a^{-1} b_a, \quad (1)$$

with $\alpha = 0.5$. Feedback is discounted by $\gamma = 0.999$ per arm update so that stale labels fade; recovering from a long drift took 144 labels instead of 793. A warm-start prior, never discounted, makes day-one behaviour sensible (redact from attack score 0.335, escalate from 0.575, block from 0.805), and a safety mask forbids *allow* at scores ≥ 0.99 whatever was learned. Labels come from reviewer verdicts on escalations, a 2% random spot-check of other decisions, and system-prompt leaks found by the output scan (weight 0.2). Table I gives the reward table.

C. Strategic Defence: DQN Session Agent

A *session* is one API key's traffic with gaps under 30 minutes. A lockout or challenge changes what happens to every later request, and the payoff arrives at the end of the session, so this is a sequential decision problem that needs reinforcement learning. Fig. 2 contrasts the two tiers.

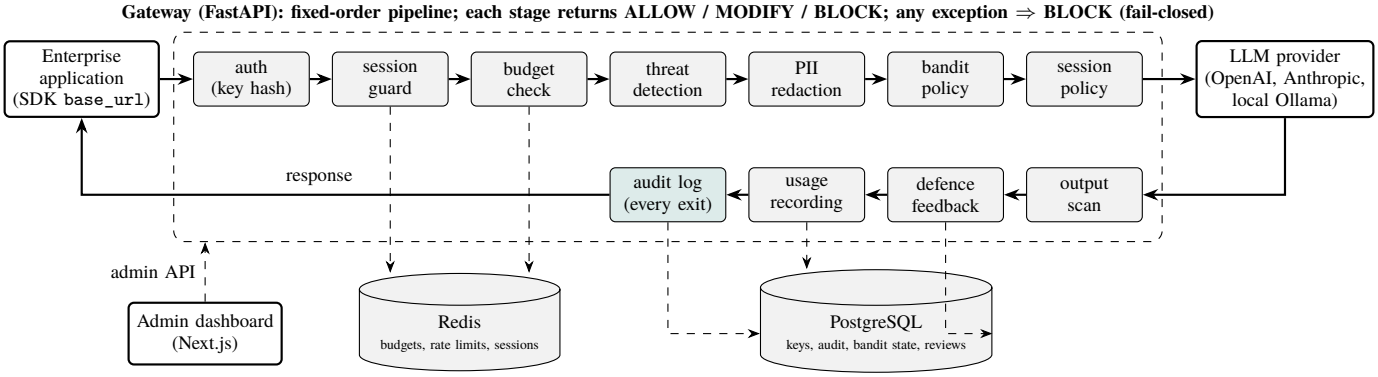


Fig. 1. Architecture and request pipeline. Solid arrows: request path; dashed: state access. The threat-detection stage only scores; the bandit and session policies decide. Audit entries are written for allowed, blocked and failed requests alike.

TABLE I
PER-REQUEST REWARD USED BY THE BANDIT

Action	Benign request	Attack request
allow	$+1.0$	-1.0
redact-and-allow	$+0.5$	0.0
block	-0.3	$+0.5$
escalate	-0.1	$+0.45$

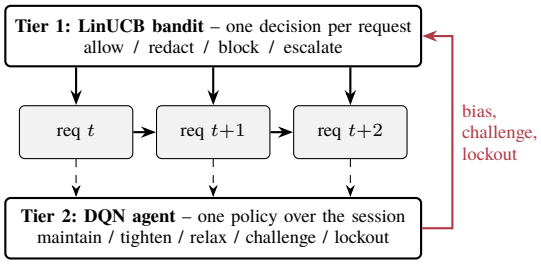


Fig. 2. Two-tier defence. The bandit decides each request independently; the session agent reads the session's history and changes how later requests are treated.

State and actions. The 15-dimensional state contains only counts, fractions and scores: the request count; the counts of allowed, redacted, blocked and escalated requests; consecutive refusals; the last, mean and maximum recent attack score and their slope; the share of requests scored ≥ 0.5; the redaction frequency; the current escalation bias; and a pending-challenge flag. It therefore cannot carry text or PII. There are five actions: *maintain*; *tighten* and *relax* (shift the bandit's escalation bias by ± 0.25, clamped to $[-0.3, 1]$); *challenge* (send the next request to human review); and *lockout* (refuse the key for 15 minutes; requires at least two refusals). We added the last two after measuring that, even at maximum bias, the bandit changes its decision for only about 1% of traffic, and that 8 of 190 held-out attacks score below 0.2, which no score-based rule can catch; only session-level actions can stop a persistent attacker who eventually lands one of these.

Reward and training. The terminal reward is $+1.0$ for a benign session (-0.5 if the agent intervened, -2.0 if it locked it out), $+1.5$ for an attack session in which it intervened and -2.0 otherwise, and -2.0 for any breach. To this we add 0.2 times the bandit reward of each request and potential-based shaping [14]

$$F(s, s') = \gamma \Phi(s') - \Phi(s), \qquad \Phi(s) = 0.5 \cdot \max_{i \leq t} p_i^{\text{attack}}. \quad (2)$$

The agent is a Stable-Baselines3 DQN [12], [13] (64–64 MLP, learning rate 5×10^{-4}, 200k–300k steps); its weights are exported to NumPy so that the gateway never loads pickled objects. A rule-based fallback was built first: tighten after two consecutive refusals and lock out after four. The training simulator replays real held-out prompts with their real classifier scores through the bandit's real code; only the session structure (lengths, probing, 85% attacker persistence, 20% false-positive-prone users) is synthetic.

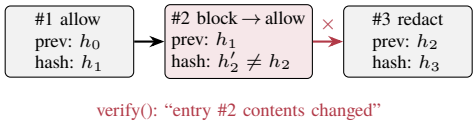


Fig. 3. Hash chain. Changing entry #2 changes its hash, which no longer matches the `prev` hash stored in entry #3; `verify` names the first broken entry.

D. Tamper-Evident Audit Log

Every exit path writes one entry with the cost, PII counts, threat score, bandit action, session action and outcome. Entries are chained as

$$h_i = \text{SHA256}\big(h_{i-1} \,\|\, \text{canon}(e_i)\big), \qquad h_0 = \text{genesis}, \quad (3)$$

so editing any entry breaks its own hash and the link from its successor (Fig. 3). Three further layers prevent the easy attacks: an ORM guard, a PostgreSQL trigger that rejects `UPDATE`, `DELETE` and `TRUNCATE`, and a restricted database role for the gateway that may only insert and read audit rows and cannot disable the trigger. The request only enqueues its entry; a single background writer chains and stores batches at about 940 entries/s.

TABLE II
DATASET SPLITS (BENIGN / INJECTION / JAILBREAK)

Split	Total	Benign	Injection	Jailbreak
v2 train	6,307	3,521	1,661	1,125
v2 validation	830	463	226	141
New test (locked)	950	546	292	112
Old test (v1)	194	100	29	65

V. EXPERIMENTAL SETUP

Data. The first dataset (v1, 1,913 prompts) combines deepset/prompt-injections, jackhhao/jailbreak-classification and benign Alpaca instructions [22]. A random split left 33 of the 91 test attacks as near-copies of training rows (jailbreak templates), which inflated injection F1 from 0.81 to 0.87. We therefore cluster near-duplicates (word 8-gram overlap $\geq$ 0.5, union-find) and assign whole clusters to one split; a re-check finds no cluster crossing splits. The expanded dataset (v2) adds the neuralchemy injection corpus, in-the-wild jailbreaks [6], NotInject hard-benign prompts [9] and Dolly instructions, removing 369 template copies. A new test set was locked by recording its SHA-256 hash before any model was trained (Table II).

Pre-registered criteria. A new classifier replaces v1 only if, on the new test, its injection F1 is higher and its benign false-positive (FP) rate is not, and on the old test its macro-F1 is at least 0.913 with an FP rate of at most 0.02. The DQN becomes the default session policy only if it beats both *maintain-only* and the rule on every scenario (paired 95% confidence interval of the return difference above zero) while locking out no more benign sessions than the rule plus one percentage point. Each model is scored once per test set.

Bandit protocol. Policies are scored by mean reward per request (Table I). Baselines are a threshold at 0.5, a threshold tuned on the *other* split, a hindsight-tuned threshold (an unachievable ceiling), and the frozen warm-start prior. The bandit runs with full feedback (an upper bound) and with realistic feedback (all escalations plus 2% spot checks). Experiment A uses i.i.d. held-out data (2 folds $\times$ 30 orders); Experiment C simulates false-positive drift by giving 20% of benign requests a score in $[0.95, 0.995]$, the band where real false positives were observed, at a 10% attack rate.

Session-agent protocol. Policies are compared on paired simulated sessions in four scenarios: *standard* (20% attack sessions), *attack-heavy* (50%), *low-rate* (5%) and *shifted* (longer sessions, more false-positive-prone users and an attacker who learns from refusals). Metrics are return, breach rate (attack sessions in which an attack was allowed) and benign-lockout rate.

VI. RESULTS

A. Classifier

Table III compares the original model (v1), the expanded model (v2) and a second attempt (v3) that started from v1's weights and oversampled v1 data to reduce forgetting. On its own 194-prompt test v1 reaches a macro-F1 of 0.933, but on the larger locked test it falls to 0.490: it flags 27.5% of benign prompts and misses most injections. v2 reaches 0.908 with a 3.1% FP rate. Over both tests together (498 attacks, 646 benign prompts) v2 misses 28 attacks against v1's 63 and flags 18 benign prompts against v1's 151.

TABLE III
CLASSIFIER RESULTS (MACRO-F1, INJECTION F1, BENIGN FP RATE)

Test set	Metric	v1	v2 (shipped)	v3
New (950)	Macro-F1	0.490	**0.908**	0.877
	Injection F1	0.341	**0.949**	0.935
	Benign FP	0.275	**0.031**	0.042
Old (194)	Macro-F1	**0.933**	0.884	0.871
	Injection F1	**0.857**	0.769	0.759
	Benign FP	**0.010**	**0.010**	0.030

TABLE IV
BANDIT RESULTS: MEAN REWARD PER REQUEST, ATTACKS ALLOWED AND BENIGN REQUESTS REFUSED

	A: i.i.d.			C: FP drift		
Policy	Reward	Att.	Ben.	Reward	Att.	Ben.
Static, tuned	0.719	4.2%	1.0%	0.697	4.7%	21.0%
Static, hindsight	–	–	–	0.905	7.6%	2.8%
Prior, frozen	0.722	4.2%	0.5%	0.703	4.7%	20.5%
Bandit, full fb.	0.714	4.2%	0.5%	**0.837**	4.7%	**0.9%**
Bandit, 2% fb.	0.721	4.2%	0.5%	0.739	4.7%	15.1%

v2 nevertheless fails the third pre-registered criterion: its old-test macro-F1 of 0.884 is below 0.913, a gap of six attacks on 194 prompts. v3 also failed. We shipped v2 as an explicitly recorded *override* of the rule, since the criterion was written to catch a regression on a small benchmark, not to keep a model that flags over a quarter of benign prompts on a five-times larger test. Single-prompt latency is 20–60 ms on the laptop GPU and 160–230 ms on CPU.

B. Contextual Bandit

Table IV summarises Experiments A and C (v1 scores). On i.i.d. data every policy is within 0.01 of the others: the bandit *ties* the tuned threshold (0.721 vs. 0.719; better in 30 of 60 runs). The scores are bimodal, with 168 of 190 attacks at ≥ 0.999 and 195 of 200 benign prompts below 0.01, so there is almost nothing in the middle for a policy to learn from, and the 4.2% of attacks the classifier scores as benign look identical to benign traffic.

Under drift the picture changes. With full feedback the bandit learns to redact rather than block the drifted band, gaining $+0.141 \pm 0.012$ reward per request over the tuned threshold in 20 of 20 runs, and benign refusals fall from 21.0% to 0.9% without letting more attacks through untouched. The frozen prior behaves like the threshold (0.703), so the gain comes from learning. The hindsight threshold scores higher only because it is tuned on the stream's own labels, and it lets 7.6% of attacks through. With the realistic 2% feedback the gain is $+0.042$, and it grows with the labelling budget (Fig. 4).

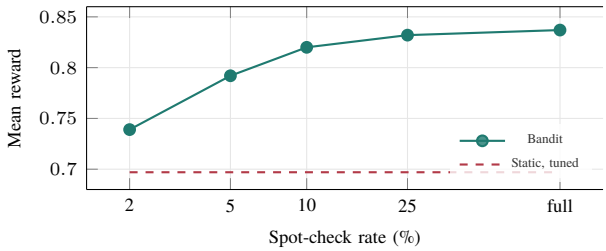


Fig. 4. Experiment C: bandit reward vs. labelling budget. Benign refusals fall from 15.1% (2%) to 7.7%, 3.5% and 1.8% (5, 10, 25%).

TABLE V
SESSION-AGENT RUNS AGAINST THE GO/NO-GO CRITERION

Run	Main change	Pass	Failure cause
1	baseline, ± shaping	2/5, 0/5	wrongful lockouts; 2 collapses
2	lockout −2.0; validation checkpoints	3/5	no gain at 5% attacks
3	v2 scores; 3 ensembles of 5 DQNs	0/3	lockout 3.1–3.5% > 2.2%

After the switch to the calibrated v2 classifier we re-ran the evaluation. The i.i.d. result is still a tie, and with full feedback the bandit still wins under drift (+0.111 on the new data). With realistic feedback, however, it now *trails* the tuned threshold at low attack rates (−0.020 at 10% and −0.024 at 2% attacks). Diagnosed on validation data, the cause is the prior: its band edges were set when scores were all-or-nothing, while 10% of v2's benign validation prompts fall between 0.3 and 0.99, and 2% feedback is too slow to move them.

C. Session Agent

Table V lists the three training runs. In run 1 every shaped seed beat both baselines on return, but three of five did so partly by locking out two to five times more benign sessions than the rule; without shaping, two seeds collapsed. Run 2 priced a wrongful lockout at −2.0, required two refusals before a lockout, and selected checkpoints on validation sessions: every seed then met the lockout limit (0.9% vs. the rule's 0.8%) and breaches fell from 14.4% to 9.9%, but two seeds were not provably better than doing nothing when only 5% of sessions are attacks. Run 3 retrained on v2 scores as three voting ensembles of five agents, evaluated on 20,000 sessions per scenario (Table VI).

The ensembles beat the rule on three scenarios and let 8.3% of attack sessions through on *standard*, against 11.7% for the rule and 16.7% for maintain-only (Fig. 5). They exceed the lockout limit, however, so the verdict is **NO-GO** and the rule remains the default; the agent is available as an opt-in. Diagnosed on validation sessions, checkpoint selection estimated lockout on only 600 sessions (about 10–15 events), so it favoured checkpoints whose lockout was low by chance: the selected agents reported 0.2–2.2% at selection but 2.2–4.8% on 3,000 fresh sessions.

TABLE VI
RUN 3: RETURN AND BENIGN-LOCKOUT RATE (20,000 SESSIONS/SCENARIO)

Scenario	Maintain	Rule	DQN ens.	Rule lock.	DQN lock.
Standard	1.569	1.779	**1.84–1.86**	1.2%	3.1–3.5%
Attack-heavy	0.390	1.257	**1.56–1.57**	1.3%	3.1–3.5%
Low-rate	**2.153**	2.040	1.97–2.00	1.2%	3.1–3.5%
Shifted	**2.220**	2.019	2.18–2.21	7.4%	6.6–7.9%

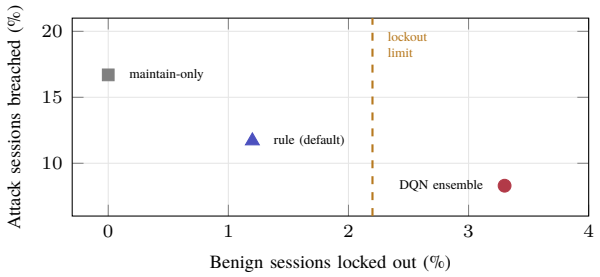


Fig. 5. Safety–security trade-off on the *standard* scenario (run 3). The DQN reduces breaches but crosses the pre-registered lockout limit.

D. End-to-End and Security Checks

The full system was tested end to end with a local Llama 3 model behind the gateway. In a prompt containing an e-mail address and a phone number, both were removed before the model saw them; the model's reply contained the literal placeholder, and the audit log held only `{EMAIL_ADDRESS:1, PHONE_NUMBER:1}`. An "ignore all previous instructions" prompt was blocked and the session tightened. Editing one audit entry from *block* to *allow* directly in PostgreSQL was detected and the entry named; under the gateway's restricted role the edit was refused outright. Dedicated boundary tests plant e-mail addresses, phone and card numbers and keys, then search every table, Redis key, debug log line, response and upstream request. These tests found three leaks that per-module tests had missed (a library logging analysed text at debug level, provider error text echoing credentials, and tool-call arguments bypassing all checks), all of which were fixed. The gateway's 437 tests pass against real PostgreSQL and Redis with 97% line-and-branch coverage, and it sustains about 9 requests/s on a laptop CPU.

VII. DISCUSSION AND LIMITATIONS

Benchmarks can mislead. The drop from 0.933 to 0.490 shows that a small, narrow test set can overstate a model badly; leak-free splits and a larger locked test were necessary to see it.

A policy adds adaptivity, not accuracy. The bandit costs nothing where the classifier is right and recovers where its errors concentrate in a score band that feedback can reveal, but it cannot fix the detector's blind spots. Separating detection from response is still valuable: it provides the safety mask, human escalation and a hook for session memory.

Downstream components inherit upstream assumptions. Better calibrated scores exposed band edges in the bandit's

prior that had been fitted, implicitly, to the old model's all-or-nothing scores.

RL found a real effect, but not reliably. The agent learns to intervene earlier in attack sessions, which the rule cannot, but its safety depended on the training seed and on a noisy selection estimate. The reward design and the statistical power of the safety constraint, rather than the algorithm, were the bottlenecks.

Limitations. Detection was evaluated only on public benchmarks, not against an adaptive attacker, and was not compared against other guard models. Very short or imperative text still triggers false positives (a bare "hello" scores 0.805), as does code in tool arguments. Some request fields (tool definitions, the `user` field) are forwarded unchecked, and the output scan detects only verbatim system-prompt leaks. The session agent was trained in a simulator with synthetic session structure, and a session is an API key, so a shared key lets one attacker lock out all its users. The reward tables are hand-designed proxies. Streaming responses are rejected so that the whole answer can be inspected first, and the deployment is a single node.

VIII. Conclusion and Future Work

We built and evaluated a fail-closed LLM gateway that combines cost control, PII redaction, injection detection, a two-tier adaptive defence and tamper-evident auditing. The classifier reaches a macro-F1 of 0.908 on a locked 950-prompt test, and the evaluation showed that a smaller benchmark had overstated an earlier model. The contextual bandit matches a tuned threshold where the classifier is right and recovers from drift where it is wrong. The DQN session agent reduced breaches relative to a rule but did not meet a pre-registered safety criterion, so the rule stays the default. Future work includes re-fitting the bandit's prior to calibrated scores, a constrained or better-powered safety criterion for the session agent trained on real logged sessions, an adaptive red-team loop, semantic leak detection, and comparison with existing guard models on the locked test set. We believe negative results reported with their causes are as useful to practitioners as positive ones.

Acknowledgment

The authors thank their guide, Mr. Monish L, and the Department of DEPARTMENT, UNIVERSITY NAME, for their support, and the authors of the public datasets used in this work.

References

[1] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. [Online]. Available: https://genai.owasp.org/llm-top-10/

[2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS ML Safety Workshop*, 2022.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec Workshop*, 2023.

[4] Y. Liu *et al.*, "Prompt injection attack against LLM-integrated applications," arXiv:2306.05499, 2023.

[5] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does LLM safety training fail?" in *Proc. NeurIPS*, 2023.

[6] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "'Do Anything Now': Characterizing and evaluating in-the-wild jailbreak prompts on large language models," in *Proc. ACM CCS*, 2024.

[7] H. Inan *et al.*, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," arXiv:2312.06674, 2023.

[8] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. EMNLP: System Demonstrations*, 2023.

[9] H. Li, X. Liu, and C. Xiao, "InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models," arXiv:2410.22770, 2024.

[10] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. WWW*, 2010, pp. 661–670.

[11] A. Garivier and E. Moulines, "On upper-confidence bound policies for switching bandit problems," in *Proc. ALT*, 2011, pp. 174–188.

[12] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[13] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[14] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. ICML*, 1999, pp. 278–287.

[15] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 8, pp. 3779–3795, 2023.

[16] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, 1999.

[17] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. USENIX Security Symp.*, 2009, pp. 317–334.

[18] K. Lee *et al.*, "Deduplicating training data makes language models better," in *Proc. ACL*, 2022, pp. 8424–8445.

[19] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *Proc. ICLR*, 2023.

[20] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321–1330.

[21] Microsoft, "Presidio: Data protection and de-identification SDK." [Online]. Available: https://github.com/microsoft/presidio

[22] R. Taori *et al.*, "Stanford Alpaca: An instruction-following LLaMA model," 2023. [Online]. Available: https://github.com/tatsu-lab/stanford_alpaca